Profile Scripting Language

PSL

What is PSL?

- Profile Scripting Language (PSL), is an object-like extension of the Mumps programming language. Standard object-oriented features (i.e. class, method and property definitions are incorporated into the PSL development environment.

Class

- A class is an entity that defines the structure and behavior (i.e. data and code) shared by a set of objects.
- Each object, therefore, has the structure and behavior defined by its class. The class defines the allowable properties of the objects in that class, as well as the methods that operate on the objects in the class.
- A class is therefore a template for an object, and an object is an instance of a class.
- Example:
  - Class: Dog
  - Object: My dog Skip
  - Properties: Hair color, age, weight, etc.

Object

- An object is an item that stores data and the operations that manipulate that data. Therefore, an object is a collection of variables and associated methods that is described by a class.
- An object is an instance of a specific class.
- An object definition consists of two components:
  - **Properties**
  - **Methods**

Object Definition Components

- **Properties**
- The properties of an object are the characteristics that describe the object. For example, the properties of a particular keyboard may be its number of keys (for example: the property of a particular keyboard (object) is it has 90 keys). The properties of an object are also called the instance variables for the object.
- Database file definitions define all properties of objects in the Record class. For example, the *DEP* file definition defines all the properties (i.e., columns) of the *RecordDEP* object.
- **Methods**
- The methods associated with an object define the operations or procedures that manipulate object data. In PSL, methods for objects defined in the *ResultSet* class include *getCol, getRow, isEmpty*, and *next*.

Class Method

- A class method can be invoked by two means:
- Assignment expression, if the method returns a value

  set cif=Db.getRecord("CIF","ACN=:ACN")
- Procedure call, if the method does not return a value

  do dep.save()

Implementing Methods

- PSL implements methods as records within the Class Methods database table, *OBJECTMET*.
- The records include a Mumps subroutine reference that is called by the PSL compiler when it encounters a method reference in the source code.
- For example, to implement the Db.getRecord method, the compiler executes the M subroutine, getRecord^UCDB.

Objects

- PSL dynamically allocates objects when they are instantiated either by calling methods that return objects or by calling the Class.new() method.
- The PSL *new* command is used to create an object within a procedure or nested block, PSL allocates memory for the object when the system processes the *new* command. PSL deallocates the memory for that object when processing exits the procedure (via the *quit* or *kill* command).
- **Example:**

  type RecordDEP dep=Class.new("RecordDEP")

  ...something

  ...something

  quit //Deallocates the variable dep and the object it points to

Declaring Objects

- The following syntax is used when declaring objects:
  type *ObjectClass identifier=expression*
- Where:
  - **type** - a required declarative keyword
  - ***ObjectClass*** - the class to which the object belongs (i.e. Record???, ResultSet, etc)
  - ***identifiers*** – the name of the object
  - ***expression*** – includes the object method and any necessary parameters
- **Example**:
  type RecordCIF cif=Class.new(“RecordCIF”)
- This declaration statement creates a RecordCIF object named *cif* of the type defined by the RecordCIF class.
  type ResultSet rs=Db.select(“ACN”,”CIF”)
- This statement declares a ResultSet object named *rs* and stores all of the customer numbers from the database in this object.

Comments

- PSL allows the use of the following formats for placing comments in the code:
- **Single Line Comments**
- // Write single line comments
  - Using this notation, only include single line comments at the end of a line of code where the line will not extend into the right margin (i.e., off the right side of the screen space). If the comment will extend into the right margin, insert the comment instead prior to the line of code that it describes.
- **Multi-Line Comments**

  /*

  Multi-line comments should be

  written using this notation.

  */

  By placing the /* and */ characters on individual lines, the comment block will be distinquished from the rest of the program code.

Commands

- In order to produce more readable code, PSL commands are written in lowercase. The following list provides a translation of the PSL commands and the associated Mumps commands:

PSL Syntax	**M Syntax**
do	D
else	E
for	F
if	I
new	N
quit	Q
set	S
write	W
while	*none*

Naming Conventions

- Adhere to the following standards when coding PSL:
- The following syntax is applied to methods:
- **Class.method()**
- **Class** - Capitalize the first letter of each word in the class name.
  - **Example:**

  if '**Class**.isValid(dep) quit

  else type **ResultSet** rs=**Db**.select("CID,BAL,LNM","DEP")
- **Method -** Use a lowercase letter for the first word in the method, followed by initial caps for each of the following words in the method name.
  - **Example:**

  type RecordDEP dep=Db.**getRecord**("DEP","CID=:CID")

  do dep.**save**()

Variables and Objects

- The following syntax is applied to variables and objects:
- **Variables** - Use all **uppercase** letters in variable names unless the variable stores a delimited string. Adhere to the Mumps programming standards for all other variables.
- **Objects** - Use all **lowercase** letters in object names. For Record objects, the object name should be identical to the table name to which it references.
- **Example:**

```
type RecordCIF cif=Db.getRecord("CIF",":ACN")
set NAM= cif.lnm
set DOB=cif.dob
set TAXID=cif.taxid

type ResultSet rs=Db.select("CID,BAL,IRN","DEP")
while rs.next() do {
        set CID=rs.getCol(1)
        set INT=rs.getCol(1)*rs.getCol(2)
}
```

Note: Do not use "v" or "V" as the first letter in object or variable names. The compiler reserves these characters to prefix object storage data structures

Separators

- To adhere to industry standards and enhance the readability of nested blocks, use curly brackets to mark the beginning and end of a block. This syntax replaces the dot notation used in M programming language.
- **Example:**

```
type ResultSet rs=Db.select("CID,BAL,IRN","DEP")
while rs.next() do {
        set CID=rs.getCol(1)
        set BAL=rs.getCol(2)
        set IRN=rs.getCol(3)
        set INTRATE=rs.getCol(2)*rs.getCol(3)
        write CID," - ",INTRATE,!
}
```

System Keywords

- PSL defines keywords for several common system entities.

Keyword	Description
%CurrentDate	Current Date
%CurrentTime	Current Time
%SystemDate	System Date
%UserID	User ID
%RoutineName	Routine Name

White Space

- When coding in PSL, use adequate whitespace to increase the readability of the code. Blank lines can be inserted between lines of executable code to make code easier to read.
- For example, insert blank lines between segments of code, before and after nested blocks, etc.).
- **Example:**

```
type RecordDEP dep=Db.getRecord(“DEP”,”:CID”)

set TYPE=dep.type
set JOB=%ProcessID

if ‘Db.isDefined(“TEST”,”JOB,TYPE”) do {
        set tmp=Class.new(“RecordTEST”)

        set tmp.job=%ProcessID
        set tmp.type=dep.type
        set tmp.totbal=tmp.totbal+dep.bal
        do tmp.save()
}
```

PSL Classes

Class	Provides a runtime description of the class methods and properties.
Db	Contains the set of all records within the database. The following methods are available.
Error	Contains properties and methods that enable programmers to identify and throw errors. The properties describe the error and the location in the code where it occurred.
IO	Provides methods and properties for data input and output
Record	Defines objects based on database table definitions. This class contains all the methods used by the database tables.
ResultSet	Defines the set of rows and columns resulting from a database selection operation.

Db Class

- The Db class contains the set of all records within the database.
- The Db class contains the following methods:

Method	Description
currVal	Returns the highest key value from the specified table.
delete	Permanently removes a record from the database.
getRecord	Retrieves a single record from the database and instantiates an object.
isDefined	Returns an indication of whether or not a record or table exists in the database.
nextVal	Returns the highest key value +1 from the specified table.
Select	Selects one or more records from a table based on the conditions specified in the WHERE clause.
selectDbSet	This method selects all columns from a table based on the conditions specified in the WHERE clause. The DbSet object can then be used as a type *Record* object.

currVal Method

- Returns the highest existing key value from a table.
- **Syntax**
- Db.currVal(*table,accessKey*)
- **Parameters**
- table-A valid database table nameaccessKeys-A comma-separated list of access keys for the table
- **Returns**
- The highest existing key value in the table.
- **Example:**

// Highest Customer Number
set ACN=**Db.currVal(“CIF”)**

//Last history sequence for the CID account
set SEQ=**Db.currVal(“HIST”,”CID=:CID”)**

delete Method

- Permanently removes a record from the database. This method is equivalent to a SQL DELETE statement.
- **Syntax**
- Db.delete(*table,whereClause*)
- **Parameters**
- table-A valid database table name
- whereClause-A valid WHERE clause
- **Example:**
- The following example deletes the last record in the UTBLBRCD table (i.e. the highest branch code).

set BRCD=Db.currVal(“UTBLBRCD”)

do **Db.delete(“UTBLBRCD”,”:BRCD”)**

getRecord Method

- This method retrieves a single record from the database and instantiates an object. Use this method to retrieve and/or modify a single record.
- **Declaration of *Record* class object**
- type Record*TABLE objectname*=Db.getRecord(…)
- **Syntax**
- Db.getRecord(*table,accessKeys,classNew*)
- **Parameters**
- table-A valid database table name.
- accessKeys-A comma-separated list of access keys for the table.
- classNew-Indicates whether a new object is instantiated if the record does not exist.
  - Valid values include:1 – instantiate an object
  - 0 – do not instantiate an object

getRecord Examples

type RecordCIF cif=**Db.getRecord("CIF",":ACN")**
set NAM=cif.nam
set DOB=cif.dob
set TAXID=cif.taxid

type RecordHIST hist
set hist=**Db.getRecord("HIST","CID=:CID,TSEQ=:TSEQ")**
set TAMT=hist.tamt
set TLO=hist.tlo
set TSO=hist.tso

getRecord vs. select & selectDbSet

- Use of the Db.select and Db.selectDbSet method is preferred over Db.getRecord if any of the following conditions exist:
  - Retrieving multiple records drom a database (*select*)
  - Retrieving one or more records based on a where clause (*select*)
  - The procedure uses *getRecord* based on information retrieved from a *select* (*selectDbSet*).

isDefined Method

- Returns a numeric value indicating whether or not a record or table exists in the database.
- **Syntax**
- Db.isDefined(*table,accessKeys*)
- **Parameters**
- table-A valid database table name.
- accessKeys-A comma-separated list of access keys for the table.
- **Returns**
  - **1**-The record or table is defined
  - **0**-The record or table is not defined
- **Example:**

if '**Db.isDefined("CIF","ACN=:ACN")** quit
else ...

nextVal Method

- Returns the highest key value +1 from the specified table. This method is only used with tables that have numeric key values.
- **Syntax**
- Db.nextVal (*table,accessKeys*)
- **Parameters**
- table-A valid database table name.
- accessKeys -A comma-separated list of access keys for the table.
- **Returns**
- Highest key value +1.
- **Examples:**

// highest Branch Code +1
set XBRCD=**Db.nextVal(“UTBLBRCD”)**

//Highest History Seq+1 for an account
set XSEQ=**Db.nextVal(“HIST”,”CID=:CID”)**

Exercise

- Prompt User for an Account Number
- Use getRecord to retrieve the Account information and write out to the screen the Balance and Status
- (ACN Data Items BAL and STAT)

ResultSet Class

- Selects one or more records from a table based on the conditions specified in the optional *where* clause and stores the information in a type *ResultSet* object.
- Use of this method is preferred if only selecting data. However, if data also needs to be updated, use the *selectDbSet* method to select and update the data.
- **Declaration of *ResultSet* class object**
- type ResultSet *rs*=Db.select(…)
- **Syntax**
- Db.select(*columnList,table,whereClause,orderByClause*)
- **Parameters**
- columnList-A comma-separated list of table columns from which to select
- table-A valid database table name
- whereClause-A valid Sql Where clause (optional)
- orderByClause -A valid Sql Order By clause (optional)
- **Returns**
- An object containing the result set from the database selection.

select Examples

Examples:

```
type ResultSet rs=Db.select("ACN,NAM,TAXID","CIF")
If rs.isEmpty() quit
while rs.next() do {
        set ACN=rs.getCol(1)
        set NAM=rs.getCol(2)
        set TAXID=rs.getCol(3)
        }

set BAL=1000
type ResultSet rs=Db.select("CID,BAL,IRN","DEP","BAL>:BAL")
If rs.isEmpty() quit
while rs.next() do {
set CID=rs.getCol(1)
set INT=(rs.getCol(2))*(rs.getCol(3))
```

Exercise

- Write out to screen all Customer Numbers stored in CIF.
- DO NOT use a direct reference to ^CIF global

Parameter Passing

- Whenever possible, specific column names or values should be passed rather than the object for PSL compiler optimization

```
    type RecordDEP dep=Db.getRecord(“DEP”,”:CID”)
    type RecordCIF cif=Db.getRecord(“CIF”,”:ACN”)
    do CHECK(dep.odt,dep.bal,dep.irn,cif.nam,cif.taxid)
    quit
CHECK(ODT,BAL,IRN,NAM,TAXID)
    set INT=BAL*IRN
    set DATA=NAM_”, “_TAXID
```

VERSUS

```
    type RecordDEP dep=Db.getRecord(“DEP”,”:CID”)
    type RecordCIF cif=Db.getRecord(“CIF”,”:ACN”)
    do CHECK(.dep,.cif)
    quit
CHECK(RecordDEP dep, RecordCIF cif )
    set BAL=dep.bal
    set NAM=cif.nam
```

selectDbSet

- Selects all columns from a table based on the conditions specified in the WHERE clause and stores the information in a type *DbSet* object. Use the *Db.getRecord* method to move the data into an object where the individual columns can be referenced.
- This method is beneficial in circumstances where *select* and *getRecord* are used in conjunction. Db.selectDbSet does not require validation that the record exists (i.e.,*Db.isDefined*) because it only returns valid records from the database.
- **Declaration of *DbSet* class object**
- type DbSet *ds*=Db.selectDbSet(…)
- **Syntax**
- Db.selectDbSet(*table,whereClause,orderByClause*)
- **Parameters**
- table-A valid database table name
- whereClause-A valid Sql Where clause (optional)
- orderByClause -A valid Sql Order By clause (optional)
- **Returns**
- An object pointer for use with the getRecord method.

selectDbSet Example

```
type DbSet ds=Db.selectDbSet("UTBLBRCD")
while ds.next() do {
    type RecordUTBLBRCD utblbrcd=ds.getRecord()
    set DESC=utblbrcd.desc
    set BRCITY=utblbrcd.brcity
    set BRSTATE=utblbrcd.brstate
    set BRZIP=utblbrcd.brzip
    }
```

IO Class

- The IO class provides methods and properties for data input and output. The methods and properties in the class are used in association with an *IO* type object; therefore, an *IO* object must first be instantiated:
- type IO *objectName*=Class.new*("IO")*
- The IO class contains the following properties:

Property	Description
directory	Defines the full path of the directory where the file is to be stored or accessed.
fileName	Defines the file name used to access the file.
openParams	Indicates the type of access granted to the device.
recordSize	Defines the length of the record within the device to be accessed
timeout	Defines the number of seconds to wait for access to the device before returning an error.

IO Class Methods

- The IO class contains the following methods:

Method	Description
close	Closes a device based on the *fileName* and *directory* properties.
open	Opens a device based on the *fileName* and *directory* properties.
read	Reads a record from the device defined by the *fileName* and *directory* properties.
write	Writes a record to the device defined by the *fileName* and *directory* properties.

directory

- This property defines the full path of the directory where the file is to be stored or accessed. If a directory is not specified, the default is the *spool* directory.
- **Syntax**
- *objectName*.directory=*directory_path*
- The following example creates a .txt file in the spool directory:
- **Example:**

type IO io=Class.new(“IO”)
set **io.directory=”/unix_directory/spool”**
set io.fileName=”TEST.txt”
set io.openParams=”NEWV/WRITE”
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

NOTE: Directory defaults to spool directory

fileName

- This property defines the filename used to access the file.
- **Syntax**
- *objectName*.fileName=*filename*
- The following example creates a .txt file in the spool directory:
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set **io.fileName=”TEST.txt”**
set io.openParams=”NEWV/WRITE”
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

openParams

- This property indicates the type of access granted to the device. Valid options include:
- NEWV – new version
- READ
- WRITE
- **Syntax**
- *objectName*.openParams=*open_parameters*
- **Parameters**
- None
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set io.fileName=”TEST.txt”
set **io.openParams=”NEWV/WRITE”**
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

recordSize

- This property defines the length of the record within the device to be accessed; the default is 512 characters. The maximum record size depends on the operating system.
- **Syntax**
- *objectName*.recordSize=*record_size*
- **Parameters**
- None
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set io.fileName=”TEST.txt”
set io.openParams=”NEWV/WRITE”
set **io.recordSize=700**
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

timeout

- This property defines the number of seconds to wait for access to the device before returning an error; the default is 1.
- **Syntax**
- *objectName.*timeout=*timeout*
- **Parameters**
- None
- **Example:**
- type IO io=Class.new(“IO”)
- set io.directory=”/unix_directory/spool”
- set io.fileName=”TEST.txt”
- set io.openParams=”NEWV/WRITE”
- set **io.timeout=5**
- do io.open()
- set TEXT=”This is a file created using IO class methods.”
- do io.write(TEXT)
- do io.close()

close

- This method closes a device based on the *fileName* and *directory* properties.
- **Syntax**
- do *objectName*.close()
- **Parameters**
- None
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set io.fileName=”TEST.txt”
set io.openParams=”NEWV/WRITE”
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

open

- This method opens a device based on the *fileName* and *directory* properties. If a directory property is not defined, the device defaults to the file name stored in the spool directory.
- **Syntax**
- do *objectName*.open()
- **Parameters**
- None; however, the *fileName* and *openParams* properties must be defined prior to using this method.
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set io.fileName=”TEST.txt”
set io.openParams=”NEWV/WRITE”
set io.timeout=5
do **io.open()**
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

read

- This method reads a record from the device defined by the *fileName* and *directory* properties. To use this method, the *openParams* property must contain “READ”.
- **Syntax**
- set *OBJ=objectName*.read(*ErrorType*)
- **Parameters**
- ErrorType - An optional parameter identifying an error plus an error description. The default ErrorType is the variable *ETYP*.
- This parameter may have the following values:
  - **0**- No Error
  - **1**- End of File
  - **2**- Device not open
- **Returns** The value read

read Example

- **Example:**

```
type IO io=Class.new("IO")
set io.directory="/unix_directory/spool"
set io.fileName="READTEST.txt"
set io.openParams="READ"
do io.open()
for  set REC=io.read() quit:+ETYP'=0  do {
    set PART1=REC.piece(",",1)
    set PART2=REC.piece(",",2)
}
do io.close()
```

write

- This method writes a record to the device defined by the *fileName* and *directory* properties. To use this method, the *openParams* property must contain “WRITE”.
- A default carriage return line feed appears after each record to separate the records. One can, however, specify a different separator as the EOLChar parameter in the write method.
- **Syntax**
- do *objectName*.write(*recordData,EOLChar*)
- **Parameters**
- recordData-Actual or variable data to write to the device. If a variable is used, this value must pass by reference.
- EOLChar-An optional end-of-line character to place after each record. The default separator is the carriage return line feed.
- **Example:**

type IO io=Class.new(“IO”)
set io.directory=”/unix_directory/spool”
set io.fileName=”TEST.txt”
set io.openParams=”NEWV/**WRITE**”
set io.timeout=5
do io.open()
set TEXT=”This is a file created using IO class methods.”
do io.write(TEXT)
do io.close()

Record Class

- The methods associated with this class are used in conjunction with type *Record* objects populated through the *Db.getRecord* method.
- The Record class defines objects based on database table definitions (for example: RecordCIF, RecordACN, RecordLN etc.). The following syntax declares a type *Record* object:

  type Record*TABLE objectname*=Db.getRecord(…)
- The properties of the objects in this class are defined by the column names in the database table definitions (for example: cif.nam, acn.bal where cif and acn are *objectnames* that belong to RecordCIF and RecordACN classes respectively.)
- The Record class contains the following methods:

Method	Description
save	Saves a record to the database.
bypassSave	Saves object data directly to the database bypassing all filers including update triggers and journal file definitions.
getMode	Determines whether the specified record exists

bypassSave

- Saves object data directly to the database bypassing all filers including update triggers and journal file definitions. Use this method for transaction processing requirements where data integrity is controlled in the program procedural logic.
- **Syntax**
- *objectName*.bypassSave()
- **Parameters**
- None
- The following example inserts a new record in the *UTBLBRCD* table. **Example:**

```
set BRCD=Db.nextVal("UTBLBRCD") //highest key value + 1
type RecordUTBLBRCD utblbrcd=Class.new("RecordUTBLBRCD")
set utblbrcd.brcd=BRCD
set utblbrcd.desc="New Branch"
set utblbrcd.brcity="San Francisco"
set utblbrcd.brstate="California"
do utblbrcd.bypassSave()
```

getMode

- This method determines whether or not the specified record exists. This method can be used in conjunction with the *Db.getRecord* method.
- When the record does not exist, the *getMode* method should be used to determine if the required field data should be added.
- **Syntax**
- *recordObject*.getMode()
- **Parameters**
- None
- **Returns**
  - **1**-If record exists
  - **0**-If record does not exist
- **Example:**

type RecordUTBLBRCD brcd=Db.getRecord(“UTBLBRCD",”:BRCD",1)
if **brcd.getMode()=0** quit

save

- Saves a record to the database using filer logic.
- **Syntax**
- *objectName*.save()
- **Parameters**
- Optional qualifiers
- **Note:** The *setAuditFlag* must be turned on in order to update the database using the *save* method.
- **Example:**

```
type RecordDEP dep=Db.getRecord(“DEP”,”:CID”)
do dep.setAuditFlag(1)
set dep.bal=dep.bal+AMT
set dep.boo=BOO
do dep.save()                    // Call DEP filer
```

DbSet Class

- The DbSet class defines the records resulting from a database selection operation using the method *Db.selectDbSet*. The methods in this class are used to retrieve data from a DbSet object.
- The following syntax declares type *DbSet* object:
- type DbSet *ds*=Db.selectDbSet(…)
- The DbSet class contains the following methods:

Method	**Description**
• getRecord	• Returns the current record
• isEmpty	• Indicates whether or not the result set contains data.
• next	• Positions the cursor at the next row of the DbSet.

DbSet getRecord

- This method is used within the DbSet class to retrieve the individual records from the DbSet object.
- **Declaration of DbSet Object**
- type DbSet ds= Db .selectDbset(…)
- **Syntax:**
- type RecordTABLE objectname=*ds.getRecord()*
- **Parameters:**
- None
- **Returns:**
- An object containing a single record based on the access key.
- **Example:**

```
type DbSet ds=Db.selectDbSet(“UTBLBRCD”)
if ds.isEmpty() quit
while ds.next() do {
    type RecordUTBLBRCD utblbrcd=ds.getRecord()
    set DESC=utblbrcd.desc
    set BRCITY=utblbrcd.brcity
    set BRSTATE=utblbrcd.brstate
    set BRZIP=utblbrcd.brzip
    }
```

DbSet isEmpty

- Indicates whether or not the DbSet contains data.
- **Syntax**
- *objectName*.isEmpty()
- **Parameters**
- None
- **Returns**
- **1**- If the resultset is empty.
- **0**- If the resultset contains data.
- **Example**

```
type DbSet ds=Db.selectDbSet(“UTBLBRCD”)
if ds.isEmpty() quit
while ds.next() do {
    type RecordUTBLBRCD utblbrcd=ds.getRecord()
    set DESC=utblbrcd.desc
    set BRCITY=utblbrcd.brcity
    set BRSTATE=utblbrcd.brstate
    set BRZIP=utblbrcd.brzip
    }
```

DbSet next

- Positions the cursor at the next row of the DbSet. When the DbSet is created, the cursor is positioned before the first row. Use this method to position the cursor on the first row.
- **Syntax**
- *objectName*.next()
- **Parameters**
- None
- **Returns**
- **1**- If more rows exist in the result set
- .**0**- If the cursor is at the end of the result set.
- **Example**

```
type DbSet ds=Db.selectDbSet("UTBLBRCD")
if ds.isEmpty() quit
while ds.next() do {
    type RecordUTBLBRCD utblbrcd=ds.getRecord()
    set DESC=utblbrcd.desc
    set BRCITY=utblbrcd.brcity
    set BRSTATE=utblbrcd.brstate
    set BRZIP=utblbrcd.brzip
    }
```

Class class

- The Class class provides a runtime description of class methods and properties.
- The Class class contains the following methods:

Method	***Description***
isValid	Returns an indication of whether or not an object is valid
New	Defines a new object in create

isValid Method

- Returns an indication of whether or not an object is valid**.**
- **Syntax**
- Class.isValid(*object*)
- **Parameters**
- object - A valid object name
- **Returns**
  - **1**- if the object is valid
  - **0**- if the object is not valid
- **Example:**
- type RecordDEP dep=Db.getRecord(“DEP”,”:CID")
- if ‘**Class.isValid(dep)** quit
- else …

new Method

- Defines a new object to create.
- **Syntax**
- Class.new(*class*)
- **Parameters**
- class - A valid class name
- **Returns**
- Object name
- **Example:**

set BRCD=Db.nextVal(“UTBLBRCD”) //highest key value + 1
type RecordUTBLBRCD utblbrcd=**Class.new(“RecordUTBLBRCD”)**
set utblbrcd.brcd=BRCD
set utblbrcd.desc=”New Branch”

Column class

- The Column class contains *properties* that enable programmers to define the attributes of a table column.
- The Column class contains the following properties:

Property	**Description**
des	Contains a description of the column
len	This property indicates the maximum length of data stored in the column
oldVal	This property enables a filer (e.g., journal, trigger) to track the old value of a column that has been changed
req	This property indicates whether the column is required
typ	This property identifies the format type of the column

des Property

- This property returns the description of the column.
- **Syntax**
- *table.column*.des
- **Parameters**
- None
- **Returns**
- A string identifying the column description
- **Example:**

type RecordDEP dep=Db.getRecord(“DEP”,”CID=:CID”)
set BALDESC=**dep.bal.des**
set IRNDESC=**dep.irn.des**

len Property

- This property indicates the static field length of the column.
- **Syntax**
- *table.column*.len
- **Parameters**
- None
- **Returns**
- A numeric value representing field length of the column
- **Example:**

type RecordCIF cif=Db.getRecord(“CIF”,”ACN=:ACN”)

if **cif.nam.len**<20 set...

oldVal Property

- This property enables a filer (e.g. trigger) to track the old value of a column that has been changed.
- **Syntax**
- *table.column*.oldVal
- **Parameters**
- None
- **Returns**
- The original value of the column before any changes occured
- **Example:**

type RecordCIF cif=Db.getRecord(“CIF”,”ACN=:ACN”)

if cif.boo=**cif.boo.oldVal**...

req Property

- This property indicates whether the column is a required field.
- **Syntax**
- *table.column*.req
- **Parameters**
- None
- **Returns**
  - **0**-Column is not required
  - **1**-Column is required
- **Example:**

type RecordDEP dep=Db.getRecord(“DEP”,”CID=:CID”)
if ‘**dep.bal.req** quit
else ...

typ Property

- This property identifies the data type of the column.
- **Syntax**
- *table.column*.typ
- **Parameters**
- None
- **Returns**
- A string representing the column data type
- **Example:**

type RecordDEP dep=Db.getRecord(“DEP”,”CID=:CID”)
set TYP=**dep.bal.typ**

ERROR Class

- The Error class contains properties and methods that enable the identification and throwing of errors. The properties describe the error and the location of the error in the code.
- The Error class contains the following properties

Property	Description
description	Optional property that provides a description of the error.
thrownAt	Identifies the runtime M location at which the error was encountered.
type	Identifies the error type from the Error system table (STBLER).

- The Error class contains the following method:

Method	Description
throw	Enables the throwing of an error from any code block or subroutine using the PSL.

description

- This optional property provides a description of the error.
- **Syntax**
- *Error*.description
- **Returns**
- The error description defined in the *throw* statement. The error description is an optional free form text description.
- **Example:**

```
catch Error {
    set ET=Error.type
    if ET["%GTM" do ZE^UTLERR quit
    set RM=Error.description
    set ET=ET_"-"_RM_"-"_Error.thrownAt
    do ^UTLERR
}
```

NOTE: UTLERR is a Utility to handle all error trapping and logging

thrownAt

- This property identifies the runtime M routine location at which the error was encountered.
- **Syntax**
- *Error*.thrownAt
- **Returns**
- The location in the routine where the error occurred
- **Example:**

```
catch Error {
    set ET=Error.type
    if ET["%GTM" do ZE^UTLERR quit
    set RM=Error.description
    set ET=ET_"-"_RM_"-"_Error.thrownAt
    do ^UTLERR
}
```

type

- This property identifies the error type from the Error system table (ER).
- **Syntax**
- set *ET*=Error.type
- **Returns**
- The error type defined in the *throw* statement If not specified, the type parameter defaults to *ETUNDE*F. The error types defined in the Error system table, *STBLER,* in Profile should be used.
- **Example:**

```
catch Error {
    set ET=Error.type
    if ET["%GTM" do ZE^UTLERR quit
    set RM=Error.description
    set ET=ET_"-"_RM_"-"_Error.thrownAt
    do ^UTLERR
}
```

throw Method

- This method enables the throwing of an error from any code block or subroutine.
- **Syntax**
- To create and throw an error, use the following syntax:
  throw Class.new("*Error*","*errorType,errorDescription*")
- **Parameters**

Error	the name of the error catch block
errorType	the error type (Error.type)
errorDescription	the error description (Error.description) (optional)

Error Example

```
catch Error {
    set ET=Error.type
    if ET["%GTM" do ZE^UTLERR quit
    set RM=Error.description
    set ET=ET_"-"_RM_"-"_Error.thrownAt
    do ^UTLERR
}
```

```
if 'Db.isDefined("DEP",":CID") ...
throw Class.new("Error","INVLDADT,Invalid account")
```

set RM=$$^MSG(*Message ID*) throw Class.new("Error",","_RM)

- Output in the SCAER Report (D ^SCAER):

03/01/2003 INVLDADT-Invalid account-ZERROR1+11^ZERROR1

Exercise

- Prompt user for Customer Number. If Customer Number does not exist, throw and log an error to SCAER

Schema Class

- The Schema class contains properties and methods associated with temporary table storage.
- The Schema class contains the following methods:

Method	Description
createTable	Creates a temporary table in Data-Qwik.
deleteTable	Removes a temporary schema from Data-Qwik.

createTable Method

- Creates a temporary table in Data-Qwik.
- **Syntax**
- Schema.createTable(*table,accessKeys,columnList,global*)
- **Parameters**
- table-The name of the temporary table to create.
- accessKeys-The access keys for the temporary table.
- columnList-A list of columns to include in the table definition.
- global-The name of the global in which to store the table.

deleteTable

- Removes a temporary table from Data-Qwik. The table that is removed *must* have been created using the *createTable* method.
- **Syntax**
- Schema.deleteTable(*table*)
- **Parameters**
- table - The name of the temporary storage table to delete.

Schema Example

```
do Schema.createTable("ZTMP","SYSDATE,CID","TYPE,BAL","ZTMP")

type ResultSet rs=Db.select("CID,TYPE,BAL","DEP")
while rs.next() do {
    type RecordZTMP ztmp=Class.new("RecordZTMP")
    set ztmp.sysdate=%SystemDate
    set ztmp.cid=rs.getCol(1)
    set ztmp.type=rs.getCol(2)
    set ztmp.bal=rs.getCol(3)

    do ztmp.bypassSave()
    }
do Schema.deleteTable(“ZTMP”)
```

String Class

- The String class contains methods used to manipulate data defined as String data type.
- The String class contains the following methods:

Method	Description
• blank	• Fills a string with blanks.
• extract	• Returns a sub-string from a given string.
• piece	• Extracts an element from a delimited string.
• zero	• Fills the string with the requisite number of zeros.

blank Method

- This method fills the string with blanks up to the specified length.
- **Syntax**
- *string*.blank(*length,just*)
- **Parameters**
- Length -the length needed to fill the variable.
- Just-justification of the string
- 0 - Left justify (Default)
- 1 - Right justify.
- **Returns**
- The string filled with blanks.
- **Example**

```
type ResultSet rs=Db.select(“NAM”,”CIF”)
while rs.next() do {
    set NAM=rs.getCol(1)
    set NAM=NAM.blank(20,1)
    }
```

extract Method

- Returns a sub-string from a given string.
- **Syntax**
- *String*.extract(*from,to*)
- **Parameters**
- from -Starting position of the sub-string to be extracted DEF=1.
- to -Ending position of the sub-string to be extracted DEF=1.

- **Returns**
- A sub-string
- **Examples:**

type RecordDEP dep=Db.getRecord(“DEP”,”:CID”)
set IRN=dep.irn
set IRN=**IRN.extract(1,2)**

piece Method

- This method extracts an element from a delimited string.
- **Syntax**
- *string*.piece(*delimiter,elementPosition*)
- **Parameters**
- delimiter-The character that is used to separate elements within a string.elementPosition -The numeric position within the string where the requested data is located. This position starts at one.
- **Note:** $C(9) is necessary in order to extract pieces from a tab-delimited string
- **Returns**
- A string sub-field of a variable.

zero Method

- This method fills the string with the requisite number of zeros as calculated from the length argument.
- **Syntax**
- *string*.zero(*length, decimal, implied*)
- **Returns**
- The string filled with zeros.
- **Example:**
- type RecordDEP dep=Db.getRecord("DEP",":CID")
- set IRN=dep.irn
- set IRN=**IRN.zero(5,2)**
- Parameters:

length	-	The length of the variable
decimal	-	Decimal precision of the variable DEF=0
implied	-	An implied logical DEF=0 0 - Decimal point provided 1 - Implied decimal (decimal is not provided)